

ThreatOS

Security Operations Platform

Complete Developer Reference — Modules 1–9

All source files with exact paths · Dependency installation · Test counts · Oracle Linux 8 deployment notes.

73

Python Files

9

Modules

435

Tests

8

DB Tables

9

API Routers

10

Frontend Hooks

Oracle Linux 8 · Python 3.11 · FastAPI · PostgreSQL/TimescaleDB · Redis · Nmap

Table of Contents

• System Requirements & Installation	
• Project Structure	
• Alembic Migrations	
• Full Stack Deployment (OL8)	
• Module 1 — Ingest Pipeline	37/37 tests
• Module 2 — Detection Rule Engine	72/72 tests
• Module 3 — Alert Service + Route	42/42 tests
• Module 4 — ATT&CK; KB + Coverage Matrix	56/56 tests
• Module 5 — Asset Registry	55/55 tests
• Module 6 — Ingest Worker	26/26 tests
• Module 7 — Attack Chain Correlation	43/43 tests
• Module 8 — Risk Scoring v2 + Purple Team	57/57 tests
• Module 9 — Nmap Scan Engine	47/47 tests

System Requirements & Installation

Oracle Linux 8 setup commands

```
# 1. Enable EPEL and CRB repositories
sudo dnf install -y epel-release
sudo dnf config-manager --set-enabled ol8_codeready_builder

# 2. Python 3.11 and build essentials
sudo dnf install -y python3.11 python3.11-pip python3.11-devel \
    gcc gcc-c++ make git curl nmap postgresql-devel openssl-devel

# 3. Docker CE
sudo dnf config-manager --add-repo \
    https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo systemctl enable --now docker
sudo usermod -aG docker $USER && newgrp docker

# 4. Project virtualenv
cd /opt/threatos
python3.11 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"

# 5. Start services
docker compose up -d db redis

# 6. Run migrations (requires live PostgreSQL)
alembic upgrade head

# 7. Verify – all 435 tests must pass
python3.11 -m pytest threatos/tests/ -v

# 8. Start API + workers
uvicorn threatos.main:app --host 0.0.0.0 --port 8000 --workers 4
python3.11 -m threatos.workers.ingest_worker &
python3.11 -m threatos.workers.coverage_worker &
```

Complete dependency list (pyproject.toml)

```
# Runtime
fastapi>=0.111.0      # Web framework
uvicorn[standard]>=0.29.0 # ASGI server
sqlalchemy[asyncio]>=2.0.30
asyncpg>=0.29.0       # PostgreSQL async driver
alembic>=1.13.0       # DB migrations
redis[hiredis]>=5.0.4  # Redis client
pydantic>=2.7.0
pydantic-settings>=2.3.0
orjson>=3.10.3        # Fast JSON
httpx>=0.27.0         # ATT&CK bundle download
stix2>=3.0.1          # ATT&CK STIX parsing
structlog>=24.1.0     # Structured logging
python-dotenv>=1.0.1
apscheduler>=3.10.4   # Coverage worker scheduler

# Dev / test only
pytest>=8.2.0
pytest-asyncio>=0.23.6
fakeredis>=2.23.0     # In-memory Redis for unit tests
aiosqlite>=0.20.0     # SQLite for integration tests (no Docker needed)
ruff>=0.4.4           # Linter

# System (install with dnf/apt, NOT pip)
# nmap – sudo dnf install nmap (OL8) / sudo apt install nmap (Ubuntu)
```

SELinux & firewall (Oracle Linux 8 only)

```
# Allow Docker bind mounts
sudo setsebool -P container_manage_cgroup 1
sudo chcon -Rt svirt_sandbox_file_t /opt/threatos

# Open internal ports
sudo firewall-cmd --permanent --add-port=8000/tcp --zone=internal
sudo firewall-cmd --permanent --add-port=5432/tcp --zone=internal
sudo firewall-cmd --permanent --add-port=6379/tcp --zone=internal
sudo firewall-cmd --reload

# Nmap SYN/UDP scans need raw socket capability
sudo setcap cap_net_raw+ep $(which nmap)
```

Project Structure

```
threatos_v2/
├── pyproject.toml          dependencies + tool config
├── docker-compose.yml      TimescaleDB + Redis
├── Dockerfile              multi-stage (dev / prod)
├── alembic.ini
├── .env.example
├── docs/oracle_linux_8_setup.md
├── threatos/
│   ├── main.py             FastAPI app + lifespan startup
│   ├── models/
│   │   ├── raw_event.py    M1 ingested log events (partitioned)
│   │   ├── detection_rule.py M2 rules with JSONB AST
│   │   ├── alert.py        M3 detection hits
│   │   ├── coverage_matrix.py M4 per-technique coverage
│   │   ├── asset.py        M5 host inventory + criticality
│   │   ├── attack_chain.py M7 correlated chains
│   │   ├── purple_team_run.py M8 purple team results
│   │   ├── scan_result.py  M9 Nmap scan results
│   │   └── core/
│   │       ├── settings.py all config from env vars
│   │       ├── database.py  async SQLAlchemy engine
│   │       ├── redis_client.py shared connection pool + publish_alert
│   │       └── attck_kb.py  M4 ATT&CK bundle in-memory cache
│   ├── ingestion/
│   │   ├── normalizer.py   M1 JSON/Syslog/CEF/WinLog → NormalizedEvent
│   │   └── syslog_parser.py M1 RFC5424 + BSD syslog
│   ├── detection/
│   │   ├── rule_ast.py     M2 11 operators, 4 node types
│   │   ├── rule_engine.py  M2 evaluate_event, compute_risk_score
│   │   ├── rule_loader.py  M2 startup rule loading from DB
│   │   └── risk_scorer.py  M8 v1 + v2 chain-aware formula
│   ├── services/
│   │   ├── alert_service.py M3 persist, filter, update status
│   │   ├── coverage_service.py M4 refresh_coverage, get_coverage
│   │   ├── asset_service.py M5 CRUD + get_criticality_for_host
│   │   ├── chain_service.py M7 correlate_alerts, idempotent upsert
│   │   ├── purple_service.py M8 validate technique/chain
│   │   └── nmap_service.py  M9 async subprocess + XML parser
│   ├── api/routers/
│   │   ├── ingest_router.py POST /api/ingest/{event,batch,syslog}
│   │   ├── rules_router.py  GET/POST /api/rules, PUT toggle
│   │   ├── alerts_router.py GET /api/alerts, PUT status
│   │   ├── coverage_router.py POST /refresh, GET matrix+summary
│   │   ├── assets_router.py GET/POST/PUT /api/assets
│   │   ├── chains_router.py POST /correlate, GET chains
│   │   ├── purple_router.py POST /validate, /score
│   │   ├── scans_router.py  POST /scans, GET results
│   │   └── ws_router.py     WS /ws/alerts (real-time feed)
│   ├── workers/
│   │   ├── ingest_worker.py M6 Redis XREADGROUP consumer
│   │   └── coverage_worker.py M4 periodic ATT&CK refresh
│   ├── migrations/versions/
│   │   ├── 001_initial.py  creates all 8 tables
│   ├── frontend/src/
│   └── lib/api.ts           typed API clients (all modules)

├── hooks/                  9 React Query hooks + WebSocket
├── tests/
│   ├── conftest.py         SQLite in-memory fixtures
│   ├── unit/               12 test files
│   └── integration/        9 test files
```

Module 1 — Ingest Pipeline

Normalises raw log events (JSON, Syslog, CEF, WinLog) into NormalizedEvent objects, fingerprints for dedup, and persists to raw_events.

Tests: 37/37 passing

Dependencies

fastapi | sqlalchemy[asyncio] | asyncpg | orjson | pydantic | aiosqlite (tests)

Bug caught by tests

Missing UniqueConstraint on hash — dedup silently failed. Caught by test_ingest_batch_deduplicates_identical_events.

Source Files

threatos/models/raw_event.py


```

        PG_UUID(as_uuid=True),
        primary_key=True,
        default=uuid.uuid4,
    )
    received_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        primary_key=True,
        nullable=False,
    )
    log_source: Mapped[str] = mapped_column(String(30), nullable=False)
    raw_payload: Mapped[dict] = mapped_column(JSONBCompat, nullable=False, default=dict)
    normalized: Mapped[dict | None] = mapped_column(JSONBCompat, nullable=True)
    hash: Mapped[str | None] = mapped_column(String(64), nullable=True, unique=True)

    __table_args__ = (
        Index("ix_raw_events_received_at", "received_at"),
        Index("ix_raw_events_log_source", "log_source"),
        UniqueConstraint("hash", name="uq_raw_events_hash"),
        {"postgresql_partition_by": "RANGE (received_at)"},
    )

    def __repr__(self) -> str:
        return (
            f"<RawEvent id={self.id} "
            f"source={self.log_source!r} "
            f"at={self.received_at}>"
        )

```

[threatos/ingestion/normalizer.py](#)


```

ingestion/syslog_parser.py
#####

Parses RFC 5424 and BSD Syslog lines into flat dicts
that the normalizer maps to NormalizedEvent fields.
"""
from __future__ import annotations

import re
from typing import Any

_RFC5424 = re.compile(
    r"<(P<pri>\d+)>(P<version>\d+)\s+"
    r"(P<timestamp>\S+)\s+(P<hostname>\S+)\s+"
    r"(P<app_name>\S+)\s+(P<procid>\S+)\s+"
    r"(P<msgid>\S+)\s+(P<structured_data>\[.*?\]|-)\s*"
    r"(P<msg>.*)",
    re.DOTALL,
)

_BSD = re.compile(
    r"<(P<pri>\d+)>(P<timestamp>\w{3}\s\d+\s+[\d:]+\s+"
    r"(P<hostname>\S+)\s+(P<tag>[^\s:]+):\s*(P<msg>.*)",
    re.DOTALL,
)

def parse_syslog(line: str) -> dict[str, Any]:
    """Parse a Syslog line into a flat dict. Raises ValueError if unparseable."""
    line = line.strip()

    m = _RFC5424.match(line)
    if m:
        d = m.groupdict()
        pri = int(d["pri"])
        return {
            "facility": pri >> 3,
            "severity": pri & 7,
            "hostname": d["hostname"] if d["hostname"] != "-" else None,
            "app_name": d["app_name"] if d["app_name"] != "-" else None,
            "procid": d["procid"] if d["procid"] != "-" else None,
            "timestamp": d["timestamp"],
            "msg": d["msg"],
            "raw": line,
            "format": "rfc5424",
        }

    m = _BSD.match(line)
    if m:
        d = m.groupdict()
        pri = int(d["pri"])
        return {
            "facility": pri >> 3,
            "severity": pri & 7,
            "hostname": d["hostname"],
            "app_name": d["tag"],
            "timestamp": d["timestamp"],
            "msg": d["msg"],
            "raw": line,
            "format": "bsd",
        }

    # Last resort – treat entire line as message body
    return {"msg": line, "raw": line, "format": "unknown"}

```


[illegible]

```
threatos/tests/integration/test_ingest_route.py
```



```

@pytest.mark.asyncio
async def test_ingest_unknown_log_source_returns_400(test_client: AsyncClient):
    resp = await test_client.post("/api/ingest/event", json={
        "log_source": "unsupported_format",
        "payload": {"key": "value"},
    })
    assert resp.status_code == 400
    assert "Unsupported log_source" in resp.json()["detail"]

@pytest.mark.asyncio
async def test_ingest_winlog_event_extract_correct_fields(
    test_client: AsyncClient,
    db_session,
):
    resp = await test_client.post("/api/ingest/event", json={
        "log_source": "winlog",
        "payload": {
            "Computer": "WIN-DC01",
            "EventData": {
                "SubjectUserName": "alice",
                "Image": "powershell.exe",
                "CommandLine": "powershell -enc abc",
            },
        },
    })
    assert resp.status_code == 202

    # Verify the normalized column was written with correct values
    result = await db_session.execute(
        text("SELECT normalized FROM raw_events LIMIT 1")
    )
    row = result.fetchone()
    assert row is not None

    import json
    normalized = json.loads(row[0])
    assert normalized["host"] == "WIN-DC01"
    assert normalized["user"] == "alice"
    assert "powershell" in normalized["process"].lower()
    assert "-enc" in normalized["command_line"]

@pytest.mark.asyncio
async def test_ingest_cef_event_returns_202(test_client: AsyncClient):

```

threatos/frontend/src/lib/api.ts


```

/**
 * frontend/src/hooks/useIngest.ts
 * ████████████████████████████████████████
 * React Query mutations for the ingest API.
 * Built against the proven /api/ingest/* routes (37 tests passing).
 *
 * Usage in any component:
 *   const { mutate, isPending, isError, data } = useIngestEvent();
 *   mutate({ log_source: "json", payload: { hostname: "srv01" } });
 */
import { useMutation } from "@tanstack/react-query";
import { ingestApi, type IngestRequest } from "@lib/api";

/**
 * Mutation hook for POST /api/ingest/event
 * Returns the IngestResponse on success (status, event_id, hash).
 */
export function useIngestEvent() {
  return useMutation({
    mutationFn: (req: IngestRequest) => ingestApi.single(req),
  });
}

/**
 * Mutation hook for POST /api/ingest/batch
 * Returns BatchIngestResponse with queued/failed counts + event_ids.
 */
export function useIngestBatch() {
  return useMutation({
    mutationFn: (events: IngestRequest[]) => ingestApi.batch(events),
  });
}

/**
 * Mutation hook for POST /api/ingest/syslog
 * Accepts a raw syslog line string.
 */
export function useIngestSyslog() {
  return useMutation({
    mutationFn: (rawLine: string) => ingestApi.syslog(rawLine),
  });
}

```

Module 2 — Detection Rule Engine

Stores detection rules as compiled AST trees (JSONB). Evaluates every ingested event against loaded rules using 11 operators across 4 node types.

Tests: 72/72 passing

Dependencies

fastapi | sqlalchemy[asyncio] | pydantic

Bug caught by tests

gen_random_uuid() unavailable in SQLite — fixed with Python-side uuid.uuid4. IntegrityError on duplicate rule name returned 500 — fixed to 409.

Source Files

threatos/models/detection_rule.py


```
confidence: Mapped[float] = mapped_column(Float, nullable=False, default=0.7)

# ■■ Tags
tags: Mapped[list] = mapped_column(JSONBCompat, nullable=False, default=list)

# ■■ Lifecycle
enabled: Mapped[bool] = mapped_column(Boolean, nullable=False, default=True)
last_triggered: Mapped[datetime | None] = mapped_column(
    DateTime(timezone=True), nullable=True
)
trigger_count: Mapped[int] = mapped_column(BigInteger, nullable=False, default=0)

__table_args__ = (
    Index("ix_detection_rules_technique_id", "technique_id"),
    Index("ix_detection_rules_tactic", "tactic"),
    Index("ix_detection_rules_enabled", "enabled"),
    UniqueConstraint("name", name="uq_detection_rules_name"),
)

def __repr__(self) -> str:
    return (
        f"<DetectionRule name={self.name!r} "
        f"technique={self.technique_id} "
        f"enabled={self.enabled}>"
    )
```

```
threatos/detection/rule_ast.py
```



```

        try:    return bool(re.search(str(self.value), val, flags))
        except re.error: return False
    case "in":    return val in [_n(i) for i in (self.value or [])]
    case "not_in": return val not in [_n(i) for i in (self.value or [])]
    case "gt":
        try:    return float(raw) > float(self.value) # type: ignore
        except (TypeError, ValueError): return False
    case "lt":
        try:    return float(raw) < float(self.value) # type: ignore
        except (TypeError, ValueError): return False
    case _: return False

class AndNode(BaseModel):
    type: Literal["and"] = "and"
    children: list["RuleNode"]
    def evaluate(self, fields: dict[str, Any]) -> bool:
        return all(c.evaluate(fields) for c in self.children)

class OrNode(BaseModel):
    type: Literal["or"] = "or"
    children: list["RuleNode"]
    def evaluate(self, fields: dict[str, Any]) -> bool:
        return any(c.evaluate(fields) for c in self.children)

class NotNode(BaseModel):
    type: Literal["not"] = "not"
    child: "RuleNode"
    def evaluate(self, fields: dict[str, Any]) -> bool:
        return not self.child.evaluate(fields)

RuleNode = Union[FieldMatch, AndNode, OrNode, NotNode]

AndNode.model_rebuild()
OrNode.model_rebuild()
NotNode.model_rebuild()

def node_from_dict(d: dict[str, Any]) -> RuleNode:
    """Reconstruct a RuleNode tree from a plain dict (loaded from JSONB)."""
    t = d.get("type")
    match t:

```

[threatos/detection/rule_engine.py](#)


```

# Intentionally module-level so workers can share it without DB re-reads.

_RULES: list[CompiledRule] = []

def load_rules_into_engine(rules_data: list[dict[str, Any]]) -> int:
    """
    Compile rule dicts (from DB rows) into CompiledRule objects.
    Stores them in the module-level cache.
    Returns the count of successfully compiled rules.

    Called once at startup by rule_loader.py.
    Thread-safe for reads; only called from a single startup coroutine.
    """
    global _RULES
    compiled: list[CompiledRule] = []
    errors = 0

    for row in rules_data:
        try:
            ast = node_from_dict(row["detection_ast"])
            compiled.append(CompiledRule(
                id=str(row["id"]),
                name=row["name"],
                technique_id=row["technique_id"],
                tactic=row["tactic"],
                log_sources=[s.lower() for s in (row.get("log_sources") or [])],
                severity=int(row["severity"]),
                confidence=float(row["confidence"]),
                ast=ast,
                tags=row.get("tags") or [],
            ))
        except Exception:
            errors += 1

    _RULES = compiled
    return len(compiled)

def get_loaded_rule_count() -> int:
    """Return the number of rules currently in the engine cache."""
    return len(_RULES)

def clear_rules() -> None:

```

[threatos/detection/rule_loader.py](#)


```

count = load_rules_into_engine(rows)

if count == 0:
    log.warning(
        "No enabled detection rules found in DB – "
        "the engine will not generate alerts until rules are created."
    )
else:
    log.info("Loaded %d detection rules into engine", count)

return count

async def reload_rules(db: AsyncSession) -> int:
    """
    Reload rules and log the delta from the previous count.
    Used by mutation routes (create, toggle) to keep the engine current.
    """
    before = get_loaded_rule_count()
    after = await load_rules(db)
    delta = after - before

    if delta > 0:
        log.info("Rule reload: +%d rules (total %d)", delta, after)
    elif delta < 0:
        log.info("Rule reload: %d rules (total %d)", delta, after)

    return after

```

[threatos/api/routers/rules_router.py](#)

[illegible]

threatos/tests/unit/test_rule_ast.py

[illegible]


```

/**
 * frontend/src/hooks/useRules.ts
 * ████████████████████████████████████████
 * React Query hooks for the detection rules API.
 * Built against proven /api/rules/* routes (109 tests passing).
 *
 * Usage:
 *   const { data: rules, isLoading } = useRules();
 *   const { mutate: createRule } = useCreateRule();
 *   const { mutate: toggleRule } = useToggleRule();
 */
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { rulesApi, type RuleCreate } from "@lib/api";

const RULES_KEY = ["rules"] as const;

/**
 * Fetch all enabled detection rules.
 * Pass enabledOnly=false to include disabled rules.
 */
export function useRules(enabledOnly = true) {
  return useQuery({
    queryKey: [...RULES_KEY, { enabledOnly }],
    queryFn: () => rulesApi.list(enabledOnly),
    staleTime: 60_000, // rules change infrequently – cache for 1 minute
  });
}

/**
 * Fetch a single rule by ID.
 * Only fetches when id is defined.
 */
export function useRule(id: string | undefined) {
  return useQuery({
    queryKey: [...RULES_KEY, id],
    queryFn: () => rulesApi.get(id!),
    enabled: id !== undefined,
  });
}

/**
 * Mutation to create a new detection rule.
 * Invalidates the rules list on success so the table refreshes.
 */
export function useCreateRule() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (rule: RuleCreate) => rulesApi.create(rule),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: RULES_KEY });
    },
  });
}

/**
 * Mutation to toggle a rule's enabled state.
 * Invalidates the rules list on success.
 */
export function useToggleRule() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (id: string) => rulesApi.toggle(id),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: RULES_KEY });
    },
  });
}

```

Module 3 — Alert Service + Route

Persists detection hits as Alert rows. Supports filtering by status, technique, risk score, host. Full status lifecycle: open→investigating→escalated→closed.

```
Tests: 42/42 passing
```

Dependencies

```
fastapi | sqlalchemy[asyncio] | pydantic
```

Source Files

```
threatos/models/alert.py
```



```
threatos/services/alert_service.py
```



```

        id=uuid.UUID(alert_dict["id"])
        if isinstance(alert_dict["id"], str) else alert_dict["id"],
        rule_id=alert_dict.get("rule_id"),
        rule_name=alert_dict.get("raw_match", {}).get("rule_name"),
        event_id=alert_dict.get("event_id"),
        technique_id=alert_dict["technique_id"],
        tactic=alert_dict["tactic"],
        severity=int(alert_dict["severity"]),
        confidence=float(alert_dict["confidence"]),
        asset_criticality=int(alert_dict.get("asset_criticality", 2)),
        risk_score=float(alert_dict["risk_score"]),
        entity_host=alert_dict.get("entity_host"),
        entity_user=alert_dict.get("entity_user"),
        entity_process=alert_dict.get("entity_process"),
        entity_ip=alert_dict.get("entity_ip"),
        status=alert_dict.get("status", "open"),
        description=alert_dict.get("description"),
        raw_match=alert_dict.get("raw_match"),
        created_at=_parse_dt(alert_dict.get("created_at")),
        updated_at=_parse_dt(alert_dict.get("updated_at")),
    )
    db.add(alert)
    await db.flush() # get the id without committing – caller controls commit
    return alert

async def persist_alerts_bulk(
    db: AsyncSession,
    alert_dicts: list[dict[str, Any]],
) -> int:
    """
    Insert multiple alerts in a single flush.
    Returns count of alerts persisted.
    Skips empty lists without hitting the DB.
    """
    if not alert_dicts:
        return 0

    alerts = []
    for d in alert_dicts:
        alert = Alert(
            id=uuid.UUID(d["id"]) if isinstance(d["id"], str) else d["id"],
            rule_id=d.get("rule_id"),
            rule_name=d.get("raw_match", {}).get("rule_name"),
            event_id=d.get("event_id"),

```

[threatos/api/routers/alerts_router.py](#)


```

    assert resp.status_code == 200
    assert resp.json() == []

@pytest.mark.asyncio
async def test_list_alerts_returns_seeded_alerts(
    test_client: AsyncClient, db_session
):
    await persist_alerts_bulk(db_session, [_make(), _make()])
    await db_session.commit()

    resp = await test_client.get("/api/alerts")
    assert resp.status_code == 200
    assert len(resp.json()) == 2

@pytest.mark.asyncio
async def test_list_alerts_response_schema(
    test_client: AsyncClient, db_session
):
    await persist_alert(db_session, _make())
    await db_session.commit()

    resp = await test_client.get("/api/alerts")
    alert = resp.json()[0]

    required_fields = {
        "id", "technique_id", "tactic", "severity", "confidence",
        "risk_score", "asset_criticality", "status",
        "entity_host", "entity_user", "created_at", "updated_at",
    }
    assert required_fields.issubset(set(alert.keys()))

@pytest.mark.asyncio
async def test_list_alerts_ordered_by_risk_score_desc(
    test_client: AsyncClient, db_session
):
    await persist_alerts_bulk(db_session, [
        _make(risk_score=20.0),
        _make(risk_score=90.0),
        _make(risk_score=55.0),
    ])
    await db_session.commit()

```

threatos/frontend/src/hooks/useAlerts.ts

```

/**
 * frontend/src/hooks/useAlerts.ts
 * ████████████████████████████████████████
 * React Query hooks for the alerts API.
 * Built against proven /api/alerts/* routes (151 tests passing).
 *
 * Usage:
 *   const { data: alerts, isLoading } = useAlerts({ status: "open" });
 *   const { mutate: updateStatus } = useUpdateAlertStatus();
 */
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import {
  alertsApi,
  type Alert,
  type AlertListParams,
  type AlertStatus,
} from "@lib/api";

const ALERTS_KEY = ["alerts"] as const;

/**
 * Fetch a filtered list of alerts.
 * Refetches every 15 seconds to catch new alerts without WebSocket.
 * (Phase 2 adds a WebSocket layer on top for sub-second updates.)
 */
export function useAlerts(params?: AlertListParams) {
  return useQuery({
    queryKey: [...ALERTS_KEY, params],
    queryFn: () => alertsApi.list(params),
    refetchInterval: 15_000,
    staleTime: 5_000,
  });
}

/**
 * Fetch a single alert by ID.
 * Skips the query when id is undefined.
 */
export function useAlert(id: string | undefined) {
  return useQuery({
    queryKey: [...ALERTS_KEY, id],
    queryFn: () => alertsApi.get(id!),
    enabled: id !== undefined,
  });
}

/**
 * Update an alert's status.
 * Optimistically updates the local cache then invalidates
 * so the list re-fetches with the new status from the server.
 */
export function useUpdateAlertStatus() {
  const queryClient = useQueryClient();

  return useMutation({

```

```

mutationFn: ({ id, status }: { id: string; status: AlertStatus }) =>
  alertsApi.updateStatus(id, status),

onMutate: async ({ id, status }) => {
  // Cancel in-flight refetches to avoid overwriting the optimistic update
  await queryClient.cancelQueries({ queryKey: ALERTS_KEY });

  // Snapshot current data for rollback on error
  const snapshot = queryClient.getQueryData(ALERTS_KEY);

  // Optimistically update matching alerts in all cached query results
  queryClient.setQueryData(
    { queryKey: ALERTS_KEY },
    (old: Alert[] | undefined) =>
      old?.map((a) => (a.id === id ? { ...a, status } : a)),
  );

  return { snapshot };
},

onError: (_err, _vars, context) => {
  // Roll back to snapshot on error
  if (context?.snapshot) {
    queryClient.setQueryData(ALERTS_KEY, context.snapshot);
  }
},

onSettled: () => {
  // Always refetch after mutation so cache matches server
  queryClient.invalidateQueries({ queryKey: ALERTS_KEY });
},
});
}

```

Module 4 — ATT&CK; KB + Coverage Matrix

Downloads the MITRE ATT&CK; STIX bundle, caches ~700 techniques in memory, and computes a per-technique coverage matrix from deployed rules.

Tests: 56/56 passing

Dependencies

stix2 | httpx | fastapi | sqlalchemy[asyncio]

Bug caught by tests

platforms stored as Python repr "[windows]" not JSON — fixed with json.dumps(). func.cast(col, NullType) crashes SQLite — fixed with case() expression.

Source Files

threatos/models/coverage_matrix.py


```

# When the last matching alert fired for this technique
last_triggered: Mapped[datetime | None] = mapped_column(
    DateTime(timezone=True), nullable=True
)

# Platforms from the ATT&CK bundle: ["windows", "linux", ...]
platforms: Mapped[list] = mapped_column(
    JSONBCompat, nullable=False, default=list
)

# True when no rule covers this technique AND it is actively
# used by tracked threat actors (Phase 3 adds actor tracking;
# Phase 1 sets this to False for all uncovered techniques)
priority_gap: Mapped[bool] = mapped_column(
    Boolean, nullable=False, default=False
)

# Timestamp of last coverage recomputation
refreshed_at: Mapped[datetime | None] = mapped_column(
    DateTime(timezone=True), nullable=True
)

__table_args__ = (
    Index("ix_coverage_tactic", "tactic"),
    Index("ix_coverage_covered", "covered"),
    Index("ix_coverage_priority", "priority_gap"),
)

def __repr__(self) -> str:
    return (
        f"<CoverageMatrix {self.technique_id} "
        f"covered={self.covered} "
        f"rules={self.rule_count}>"
    )

```

[threatos/core/attck_kb.py](#)


```

if not all_tids:
    return CoverageSummary(total_techniques=0, covered=0, gaps=0, coverage_pct=0.0)

# 1. Rule stats per technique
rule_stats_q = (
    select(
        DetectionRule.technique_id,
        func.count(DetectionRule.id).label("cnt"),
        func.avg(DetectionRule.confidence).label("avg_conf"),
    )
    .where(DetectionRule.enabled.is_(True))
    .group_by(DetectionRule.technique_id)
)
rule_result = await db.execute(rule_stats_q)
rule_stats: dict[str, dict[str, Any]] = {
    row.technique_id: {
        "count": int(row.cnt),
        "avg_conf": float(row.avg_conf or 0.0),
    }
    for row in rule_result
}

# 2. Last alert per technique
last_alert_q = (
    select(
        Alert.technique_id,
        func.max(Alert.created_at).label("last_fired"),
    )
    .group_by(Alert.technique_id)
)
last_result = await db.execute(last_alert_q)
last_fired: dict[str, datetime] = {
    row.technique_id: row.last_fired for row in last_result
}

# 3. Upsert all technique rows
covered_count = 0
gap_count = 0

for tid in all_tids:
    tech = get_technique(tid)
    stats = rule_stats.get(tid, {"count": 0, "avg_conf": 0.0})
    is_covered = stats["count"] > 0

```

threatos/api/routers/coverage_router.py


```
... (showing first 100 of 125 lines)
```

```

        if attempt < 3:
            await asyncio.sleep(5 * attempt)
        log.error("Could not load ATT&CK bundle after 3 attempts – "
            "coverage matrix will remain empty until next interval")
    return 0

async def _do_refresh() -> None:
    """Run one full coverage matrix refresh cycle."""
    if get_cache_size() == 0:
        log.info("ATT&CK cache empty – attempting bundle reload before refresh")
        await _load_attck()

    if get_cache_size() == 0:
        log.warning("Skipping coverage refresh: no ATT&CK techniques in cache")
        return

    try:
        async with get_db_context() as db:
            summary = await refresh_coverage(db)
        log.info(
            "Coverage refreshed – total=%d covered=%d gaps=%d pct=%.1f%%",
            summary.total_techniques,
            summary.covered,
            summary.gaps,
            summary.coverage_pct,
        )
    except Exception as exc:
        log.error("Coverage refresh failed: %s", exc)

async def run_coverage_worker() -> None:
    """
    Main loop. Downloads ATT&CK at startup then refreshes on interval.
    Graceful shutdown on SIGTERM / SIGINT.
    """
    signal.signal(signal.SIGTERM, _handle_signal)
    signal.signal(signal.SIGINT, _handle_signal)

    log.info(
        "Coverage worker starting – interval=%ds",
        settings.coverage_refresh_interval_seconds,
    )

    # Initial load + refresh

```

[threatos/tests/unit/test_attck_kb.py](#)


```
... (showing first 100 of 257 lines)
```

threatos/tests/integration/test_coverage_route.py


```
assert "gaps" in data
assert "coverage_pct" in data
```

```
@pytest.mark.asyncio
async def test_refresh_total_matches_kb_size(test_client: AsyncClient):
    resp = await test_client.post("/api/coverage/refresh")
    assert resp.json()["total_techniques"] == len(SAMPLE_KB)
```

```
@pytest.mark.asyncio
async def test_refresh_all_gaps_when_no_rules(test_client: AsyncClient):
    resp = await test_client.post("/api/coverage/refresh")
    data = resp.json()
    assert data["covered"] == 0
    assert data["gaps"] == len(SAMPLE_KB)
    assert data["coverage_pct"] == 0.0
```

```
@pytest.mark.asyncio
async def test_refresh_counts_covered_correctly(
    test_client: AsyncClient, db_session
):
    db_session.add(_rule("Rule PS", "T1059.001"))
    await db_session.commit()

    resp = await test_client.post("/api/coverage/refresh")
    data = resp.json()
    assert data["covered"] == 1
    assert data["gaps"] == 2
    assert data["coverage_pct"] == pytest.approx(33.3, abs=0.2)
```

```
@pytest.mark.asyncio
async def test_refresh_is_idempotent(test_client: AsyncClient):
    resp1 = await test_client.post("/api/coverage/refresh")
    resp2 = await test_client.post("/api/coverage/refresh")
    assert resp1.json() == resp2.json()
```

```
# GET /api/coverage
```

```
@pytest.mark.asyncio
async def test_get_coverage_empty_before_refresh(test_client: AsyncClient):
    resp = await test_client.get("/api/coverage")
```

threatos/frontend/src/hooks/useCoverage.ts

```

/**
 * frontend/src/hooks/useCoverage.ts
 * ████████████████████████████████████████
 * React Query hooks for the ATT&CK coverage API.
 * Built against proven /api/coverage/* routes (207 tests passing).
 *
 * Usage:
 *   const { data: rows }    = useCoverageMatrix();
 *   const { data: summary } = useCoverageSummary();
 *   const { mutate: refresh, isPending } = useRefreshCoverage();
 */
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { coverageApi, type CoverageListParams } from "@Lib/api";

const COVERAGE_KEY = ["coverage"] as const;
const SUMMARY_KEY = ["coverage", "summary"] as const;

/**
 * Fetch the full ATT&CK coverage matrix.
 * Stale after 5 minutes – coverage changes only when rules change.
 */
export function useCoverageMatrix(params?: CoverageListParams) {
  return useQuery({
    queryKey: [...COVERAGE_KEY, params],
    queryFn: () => coverageApi.getMatrix(params),
    staleTime: 5 * 60_000,
  });
}

/**
 * Fetch coverage aggregate stats for the dashboard header.
 * Lighter than the full matrix – just totals.
 */
export function useCoverageSummary() {
  return useQuery({
    queryKey: SUMMARY_KEY,
    queryFn: coverageApi.getSummary,
    staleTime: 2 * 60_000,
    refetchInterval: 2 * 60_000,
  });
}

/**
 * Trigger a full coverage matrix recomputation.
 * Invalidates all coverage queries on success so the UI refreshes.
 */
export function useRefreshCoverage() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: coverageApi.refresh,
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: COVERAGE_KEY });
    },
  });
}

```

```

/**
 * Convenience: group coverage rows by tactic for heatmap rendering.
 * Returns a Map<tactic, CoverageRow[]> from any coverage row array.
 */
export function groupByTactic(
  rows: ReturnType<typeof useCoverageMatrix>["data"],
): Map<string, NonNullable<typeof rows>[number][]> {
  const map = new Map<string, NonNullable<typeof rows>[number][]>();
  if (!rows) return map;
  for (const row of rows) {
    const t = row.tactic ?? "unknown";
    if (!map.has(t)) map.set(t, []);
    map.get(t)!.push(row);
  }
  return map;
}

```

Module 5 — Asset Registry

Stores enriched host records with criticality tiers (1-4). The criticality value multiplies directly into the risk score formula via `get_criticality_for_host()`.

Tests: 55/55 passing

Dependencies

fastapi | sqlalchemy[asyncio] | pydantic

Source Files

threatos/models/asset.py

[illegible]

[illegible]

```
threatos/tests/unit/test_asset_service.py
```



```

@pytest.mark.asyncio
async def test_create_asset_strips_hostname_whitespace(db_session):
    asset = await create_asset(db_session, _data(hostname=" server01 "))
    assert asset.hostname == "server01"

@pytest.mark.asyncio
async def test_create_asset_returns_correct_criticality(db_session):
    asset = await create_asset(db_session, _data(criticality=4))
    assert asset.criticality == 4

@pytest.mark.asyncio
async def test_create_asset_invalid_criticality_raises(db_session):
    with pytest.raises(ValueError, match="criticality"):
        await create_asset(db_session, _data(criticality=5))

@pytest.mark.asyncio
async def test_create_asset_invalid_environment_raises(db_session):
    data = _data()
    data.environment = "INVALID_ENV"
    with pytest.raises(ValueError, match="environment"):
        await create_asset(db_session, data)

@pytest.mark.asyncio
async def test_create_asset_all_valid_criticalities(db_session):
    for tier in (1, 2, 3, 4):
        asset = await create_asset(db_session, _data(
            hostname=f"host-crit-{tier}", criticality=tier
        ))
        assert asset.criticality == tier

@pytest.mark.asyncio
async def test_create_asset_all_valid_environments(db_session):
    from threatos.models.asset import VALID_ENVIRONMENTS
    for i, env in enumerate(VALID_ENVIRONMENTS):
        asset = await create_asset(db_session, _data(
            hostname=f"host-env-{i}", environment=env
        ))
        assert asset.environment == env

@pytest.mark.asyncio
async def test_create_asset_with_ip_addresses(db_session):
    data = _data()
    data.ip_addresses = ["10.0.0.1", "192.168.1.100"]
    asset = await create_asset(db_session, data)
    assert "10.0.0.1" in asset.ip_addresses

```

[threatos/tests/integration/test_assets_route.py](#)


```

):
    await create_asset(db_session, AssetCreate(hostname="schema-host"))
    await db_session.commit()

    asset = (await test_client.get("/api/assets")).json()[0]
    required = {"id", "hostname", "criticality", "environment", "os_type", "tags", "ip_addresses"}
    assert required.issubset(set(asset.keys()))

@pytest.mark.asyncio
async def test_list_assets_ordered_criticality_desc(
    test_client: AsyncClient, db_session
):
    for tier, host in [(1, "h1"), (4, "h4"), (2, "h2"), (3, "h3")]:
        await create_asset(db_session, AssetCreate(hostname=host, criticality=tier))
    await db_session.commit()

    crits = [a["criticality"] for a in (await test_client.get("/api/assets")).json()]
    assert crits == sorted(crits, reverse=True)

@pytest.mark.asyncio
async def test_list_assets_filter_criticality(
    test_client: AsyncClient, db_session
):
    await create_asset(db_session, AssetCreate(hostname="h4", criticality=4))
    await create_asset(db_session, AssetCreate(hostname="h2", criticality=2))
    await db_session.commit()

    resp = await test_client.get("/api/assets?criticality=4")
    assert len(resp.json()) == 1
    assert resp.json()[0]["criticality"] == 4

@pytest.mark.asyncio
async def test_list_assets_filter_environment(
    test_client: AsyncClient, db_session
):
    await create_asset(db_session, AssetCreate(hostname="prod1", environment="production"))
    await create_asset(db_session, AssetCreate(hostname="dev1", environment="development"))
    await db_session.commit()

    resp = await test_client.get("/api/assets?environment=production")
    assert len(resp.json()) == 1

```

threatos/frontend/src/hooks/useAssets.ts

```

/**
 * frontend/src/hooks/useAssets.ts
 * ████████████████████████████████████████
 * React Query hooks for the asset registry API.
 * Built against proven /api/assets/* routes (262 tests passing).
 *
 * Usage:
 *   const { data: assets }    = useAssets();
 *   const { mutate: create }  = useCreateAsset();
 *   const { mutate: setCrit } = useUpdateCriticality();
 */
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import {
  assetsApi,
  type Asset,
  type AssetCreatePayload,
  type AssetListParams,
} from "@lib/api";

const ASSETS_KEY = ["assets"] as const;

/** Fetch the full asset list with optional filters. */
export function useAssets(params?: AssetListParams) {
  return useQuery({
    queryKey: [...ASSETS_KEY, params],
    queryFn: () => assetsApi.list(params),
    staleTime: 60_000, // assets change infrequently
  });
}

/** Fetch a single asset by ID. */
export function useAsset(id: string | undefined) {
  return useQuery({
    queryKey: [...ASSETS_KEY, id],
    queryFn: () => assetsApi.get(id!),
    enabled: id !== undefined,
  });
}

/**
 * Create a new asset.
 * Returns 409 if hostname already exists – callers must handle the error.
 */
export function useCreateAsset() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (data: AssetCreatePayload) => assetsApi.create(data),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ASSETS_KEY }),
  });
}

/**
 * Create-or-update an asset by hostname.
 * Safer than useCreateAsset when the hostname may already exist.
 */

```

... (showing first 100 of 101 lines)

```

export function useUpsertAsset() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (data: AssetCreatePayload) => assetsApi.upsert(data),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ASSETS_KEY }),
  });
}

/**
 * Update the criticality tier of an existing asset.
 * Optimistically updates the cached list so the UI responds instantly.
 */
export function useUpdateCriticality() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({
      id,
      criticality,
    }: { id: string; criticality: 1 | 2 | 3 | 4 }) =>
      assetsApi.updateCriticality(id, criticality),

    onMutate: async ({ id, criticality }) => {
      await queryClient.cancelQueries({ queryKey: ASSETS_KEY });
      const snapshot = queryClient.getQueriesData({ queryKey: ASSETS_KEY });

      queryClient.setQueriesData(
        { queryKey: ASSETS_KEY },
        (old: Asset[] | undefined) =>
          old?.map((a) => (a.id === id ? { ...a, criticality } : a)),
      );
      return { snapshot };
    },

    onError: (_err, _vars, ctx) => {
      if (ctx?.snapshot) {
        for (const [key, data] of ctx.snapshot) {
          queryClient.setQueryData(key, data);
        }
      }
    },

    onSettled: () =>
      queryClient.invalidateQueries({ queryKey: ASSETS_KEY }),
  });
}

```

Module 6 — Ingest Worker

Redis Streams consumer (XREADGROUP). Reads normalised events, runs them through the rule engine, resolves asset criticality (Redis-cached), persists alerts in bulk, then XACK.

Tests: 26/26 passing

Dependencies

```
redis[hiredis] | fakeredis[aioredis] (tests) | asyncio
```

Bug caught by tests

`_resolve_criticality` hit real PostgreSQL on cache miss in unit tests — fixed by pre-seeding fake cache. Wrong async context manager mock pattern in integration tests.

Source Files

```
threatos/workers/ingest_worker.py
```


[illegible]

threatos/core/redis_client.py


```

"""
Return the shared Redis client singleton.
Created once per process; lru_cache ensures only one instance exists.
Workers call this instead of creating their own connections.
"""
return _make_client()

async def check_redis_health() -> bool:
    """
    Ping Redis. Returns True if reachable, False otherwise.
    Used by the /health endpoint (Phase 2).
    """
    try:
        client = get_redis_client()
        await client.ping()
        return True
    except Exception as exc:
        log.warning("Redis health check failed: %s", exc)
        return False

async def publish_alert(alert_dict: dict) -> None:
    """
    Publish a new alert to the threatos:alerts:live pubsub channel.
    Called by the ingest worker after persisting each batch.
    The WebSocket router (ws_router.py) subscribes to this channel
    and fans the message to connected browser clients.
    """
    import json
    try:
        client = get_redis_client()
        await client.publish(
            "threatos:alerts:live",
            json.dumps(alert_dict),
        )
    except Exception as exc:
        log.warning("Failed to publish alert to Redis pubsub: %s", exc)

async def get_stream_lag(stream: str, group: str) -> int:
    """
    Return the number of unprocessed messages in a consumer group.
    Useful for monitoring the ingest worker backlog.
    Returns -1 if the stream or group does not exist.

```

[threatos/core/settings.py](#)

[illegible]

```
threatos/tests/unit/test_ingest_worker.py
```



```

return fake_aioredis.FakeRedis(decode_responses=True)

@pytest.fixture
def cache(fake_redis):
    return _CriticalityCache(fake_redis)

def _powershell_fields() -> dict:
    """Stream message fields for a PowerShell -EncodedCommand event."""
    return {
        "event_id": str(uuid.uuid4()),
        "log_source": "winlog",
        "host": "WIN-VICTIM",
        "user": "alice",
        "process": "powershell.exe",
        "command_line": "powershell.exe -EncodedCommand SQBFAFGA",
        "raw_fields": json.dumps({
            "process": "powershell.exe",
            "command_line": "powershell.exe -EncodedCommand SQBFAFGA",
        })
    }

def _load_ps_rule():
    """Load a single PowerShell detection rule into the engine."""
    load_rules_into_engine([
        {
            "id": "rule-ps-001",
            "name": "PowerShell encoded command",
            "technique_id": "T1059.001",
            "tactic": "execution",
            "log_sources": [],
            "severity": 7,
            "confidence": 0.85,
            "detection_ast": {
                "type": "and",
                "children": [
                    {
                        "type": "field_match", "field": "process",
                        "operator": "contains", "value": "powershell"
                    },
                    {
                        "type": "field_match", "field": "command_line",
                        "operator": "contains", "value": "-EncodedCommand"
                    }
                ]
            },
            "tags": ["attack.t1059.001"],
        }
    ])

```

`threatos/tests/integration/test_ingest_worker_integration.py`


```

def _fields(
    log_source: str = "winlog",
    process: str = "powershell.exe",
    cmdline: str = "powershell.exe -EncodedCommand SQBFAFGA",
    host: str = "WIN-VICTIM",
) -> dict:
    return {
        "event_id": str(uuid.uuid4()),
        "log_source": log_source,
        "host": host,
        "process": process,
        "command_line": cmdline,
        "raw_fields": json.dumps({"process": process, "command_line": cmdline}),
    }

def _load_ps_rule(log_sources: list | None = None) -> None:
    load_rules_into_engine([
        {
            "id": f"rule-{uuid.uuid4()}",
            "name": "PowerShell encoded command",
            "technique_id": "T1059.001",
            "tactic": "execution",
            "log_sources": log_sources or [],
            "severity": 7,
            "confidence": 0.85,
            "detection_ast": {
                "type": "and",
                "children": [
                    {
                        "type": "field_match", "field": "process",
                        "operator": "contains", "value": "powershell",
                    },
                    {
                        "type": "field_match", "field": "command_line",
                        "operator": "contains", "value": "-EncodedCommand",
                    },
                ],
            },
            "tags": [],
        }
    ])

async def _push_and_read(redis, fields: dict, count: int = 10):
    await redis.xadd(STREAM, fields)
    raw = await redis.xreadgroup(GROUP, "test-worker",
                                streams={STREAM: ">"}, count=count)

    return raw[0][1]

```

[threatos/frontend/src/hooks/useAlertStream.ts](#)

```
/**
 * frontend/src/hooks/useAlertStream.ts
 * ████████████████████████████████████████
 * WebSocket hook for real-time alert feed.
 *
 * The ingest worker publishes to the Redis pubsub channel
 * "threatos:alerts:live" after each batch. A FastAPI WebSocket
 * endpoint (Phase 2, ws_router.py) subscribes to that channel and
 * fans the messages out to connected browser clients.
 *
 * This hook manages the WebSocket lifecycle and hands new alerts
 * to React Query's cache so the useAlerts hook auto-refreshes.
 *
 * Usage:
 * // In your root component (e.g. App.tsx):
 * useAlertStream(); // connect once at app level
 *
 * // In a dashboard panel:
 * const { data: alerts } = useAlerts(); // auto-updated by stream
 */
import { useEffect, useRef } from "react";
import { useQueryClient } from "@tanstack/react-query";
import type { Alert } from "@lib/api";

const WS_URL = (
  window.location.protocol === "https:" ? "wss://" : "ws://" ) +
  window.location.host +
  "/ws/alerts"
);

/** How long to wait before attempting to reconnect (ms). */
const RECONNECT_DELAY_MS = 3_000;

/**
 * Connect to the alert WebSocket stream.
 * Inserts incoming alerts into the React Query cache so all
 * `useAlerts()` subscribers see them without a full re-fetch.
 * Reconnects automatically on disconnect.
 */
export function useAlertStream(): void {
  const queryClient = useQueryClient();
  const wsRef = useRef<WebSocket | null>(null);
  const reconnectRef = useRef<ReturnType<typeof setTimeout> | null>(null);

  useEffect(() => {
    let destroyed = false;

    function connect() {
      if (destroyed) return;

      const ws = new WebSocket(WS_URL);
      wsRef.current = ws;

      ws.onmessage = (event) => {
        try {
```

```

    const alert: Alert = JSON.parse(event.data);

    // Prepend the new alert into every cached alerts list
    queryClient.setQueriesData<Alert[]>(
      { queryKey: ["alerts"] },
      (old) => (old ? [alert, ...old] : [alert]),
    );
  } catch {
    // Ignore unparseable messages
  }
};

ws.onclose = () => {
  if (!destroyed) {
    reconnectRef.current = setTimeout(connect, RECONNECT_DELAY_MS);
  }
};

ws.onerror = () => {
  ws.close(); // triggers onclose → reconnect
};
}

connect();

return () => {
  destroyed = true;
  if (reconnectRef.current) clearTimeout(reconnectRef.current);
  if (wsRef.current) wsRef.current.close();
};
}, [queryClient]);
}

```

Module 7 — Attack Chain Correlation

Correlates alerts on a single host within a time window into multi-stage attack chains. Idempotent: same alert set always produces the same chain_hash.

Tests: 43/43 passing

Dependencies

fastapi | sqlalchemy[asyncio] | pydantic

Source Files

threatos/models/attack_chain.py


```
threatos/api/routers/chains_router.py
```


[illegible]

threatos/tests/unit/test_chain_service.py


```
#  _sort_tactics 
```

```
# correlate_alerts
```



```
# █ GET /api/chains █
```

[threatos/frontend/src/hooks/useChains.ts](#)

```

/**
 * frontend/src/hooks/useChains.ts
 * ████████████████████████████████████████
 * React Query hooks for the attack chain API.
 * Built against proven /api/chains/* routes (331 tests passing).
 *
 * Usage:
 * const { data: chains } = useChains({ is_multi_stage: true });
 * const { mutate: correlate } = useCorrelate();
 * const { mutate: setStatus } = useUpdateChainStatus();
 */
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import {
  chainsApi,
  type AttackChain,
  type ChainListParams,
  type ChainStatus,
  type CorrelateRequest,
} from "@lib/api";

const CHAINS_KEY = ["chains"] as const;

/** Fetch attack chains with optional filters. */
export function useChains(params?: ChainListParams) {
  return useQuery({
    queryKey: [...CHAINS_KEY, params],
    queryFn: () => chainsApi.list(params),
    refetchInterval: 30_000, // chains change as new alerts arrive
    staleTime: 10_000,
  });
}

/** Fetch a single chain by ID. */
export function useChain(id: string | undefined) {
  return useQuery({
    queryKey: [...CHAINS_KEY, id],
    queryFn: () => chainsApi.get(id!),
    enabled: id !== undefined,
  });
}

/**
 * Trigger correlation for one host.
 * Invalidates the chains list on success.
 */
export function useCorrelate() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (req: CorrelateRequest) => chainsApi.correlate(req),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: CHAINS_KEY }),
  });
}

/**
 * Update the status of an attack chain.

```

```

    * Optimistically patches the list cache then invalidates.
    */
export function useUpdateChainStatus() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ id, status }: { id: string; status: ChainStatus }) =>
      chainsApi.updateStatus(id, status),

    onMutate: async ({ id, status }) => {
      await queryClient.cancelQueries({ queryKey: CHAINS_KEY });
      const snapshot = queryClient.getQueriesData({ queryKey: CHAINS_KEY });
      queryClient.setQueriesData(
        { queryKey: CHAINS_KEY },
        (old: AttackChain[] | undefined) =>
          old?.map((c) => (c.id === id ? { ...c, status } : c)),
      );
      return { snapshot };
    },

    onError: (_err, _vars, ctx) => {
      if (ctx?.snapshot) {
        for (const [key, data] of ctx.snapshot) {
          queryClient.setQueryData(key, data);
        }
      }
    },

    onSettled: () =>
      queryClient.invalidateQueries({ queryKey: CHAINS_KEY }),
  });
}

```

Module 8 — Risk Scoring v2 + Purple Team

v2 risk formula adds chain context: each additional tactic raises score by 15%; multi-stage chains add 25% on top. Purple team validates which rules fire against emulated events.

Tests: 57/57 passing

Dependencies

fastapi | sqlalchemy[asyncio] | pydantic

Bug caught by tests

Timezone-aware vs naive datetime comparison in ordering test — fixed to use timezone-naive datetimes matching SQLite behaviour.

Source Files

threatos/models/purple_team_run.py


```

rules_fired: Mapped[list] = mapped_column(JSONBCompat, nullable=False, default=list)
rules_missed: Mapped[list] = mapped_column(JSONBCompat, nullable=False, default=list)
detection_rate: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
verdict: Mapped[str] = mapped_column(String(10), nullable=False, default="unknown" ...)

# Context
run_by: Mapped[str | None] = mapped_column(String(100), nullable=True)
notes: Mapped[str | None] = mapped_column(String(1000), nullable=True)
run_at: Mapped[datetime] = mapped_column(
    DateTime(timezone=True), nullable=False
)

def __repr__(self) -> str:
    return (
        f"<PurpleTeamRun technique={self.technique_id} "
        f"verdict={self.verdict!r} "
        f"detection={self.detection_rate:.0f}%>"
    )

```

`threatos/detection/risk_scorer.py`

[illegible]

[illegible]

```
threatos/tests/unit/test_risk_scorer.py
```



```

    assert v2.final_score == v1

def test_v2_chain_boost_increases_score():
    standalone = compute_score_v2(7, 0.85, 2, chain_tactic_count=1)
    in_chain = compute_score_v2(7, 0.85, 2, chain_tactic_count=4)
    assert in_chain.final_score > standalone.final_score

def test_v2_multi_stage_boost_increases_score():
    single_stage = compute_score_v2(7, 0.85, 2, chain_tactic_count=3, is_multi_stage=False)
    multi_stage = compute_score_v2(7, 0.85, 2, chain_tactic_count=3, is_multi_stage=True)
    assert multi_stage.final_score > single_stage.final_score

def test_v2_multi_stage_boost_is_125_percent():
    a = compute_score_v2(7, 0.85, 2, chain_tactic_count=3, is_multi_stage=False)
    b = compute_score_v2(7, 0.85, 2, chain_tactic_count=3, is_multi_stage=True)
    assert abs(b.final_score - a.final_score * 1.25) < 0.1

def test_v2_tactic_step_is_15_percent_per_tactic():
    base = compute_score_v2(7, 0.85, 2, chain_tactic_count=1).base_score
    two_t = compute_score_v2(7, 0.85, 2, chain_tactic_count=2)
    assert abs(two_t.chain_boost - 1.15) < 0.001

def test_v2_chain_boost_caps_at_2x():
    very_long = compute_score_v2(7, 0.85, 2, chain_tactic_count=100)
    assert very_long.chain_boost == 2.0

def test_v2_final_score_never_exceeds_100():
    score = compute_score_v2(10, 1.0, 4, chain_tactic_count=14, is_multi_stage=True)
    assert score.final_score <= 100.0

def test_v2_all_components_in_result():
    result = compute_score_v2(7, 0.85, 2, chain_tactic_count=3, is_multi_stage=True)
    d = result.as_dict()
    required = {
        "severity", "confidence", "asset_criticality",
        "chain_tactic_count", "is_multi_stage",
        "base_score", "chain_boost", "stage_boost", "final_score",
    }
    assert required == set(d.keys())

def test_v2_severity_clamped_to_valid_range():
    # severity=0 should be clamped to 1
    r = compute_score_v2(0, 0.8, 2)
    assert r.severity == 1
    # severity=99 clamped to 10

```

`threatos/tests/unit/test_purple_service.py`

[illegible]


```

/**
 * frontend/src/hooks/usePurple.ts
 * ████████████████████████████████████████
 * React Query hooks for the purple team and risk scoring API.
 * Built against proven /api/purple/* routes (388 tests passing).
 */
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import {
  purpleApi,
  type ScoreV2Request,
  type ValidateChainRequest,
  type ValidateRequest,
} from "@lib/api";

const RUNS_KEY = ["purple-runs"] as const;

/** Fetch purple team run history. */
export function usePurpleRuns(params?: { technique_id?: string; chain_id?: string }) {
  return useQuery({
    queryKey: [...RUNS_KEY, params],
    queryFn: () => purpleApi.runs(params),
    staleTime: 30_000,
  });
}

/** Run a purple team validation for one technique. */
export function useValidateTechnique() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (req: ValidateRequest) => purpleApi.validate(req),
    onSuccess: () => qc.invalidateQueries({ queryKey: RUNS_KEY }),
  });
}

/** Validate all techniques in an attack chain. */
export function useValidateChain() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (req: ValidateChainRequest) => purpleApi.validateChain(req),
    onSuccess: () => qc.invalidateQueries({ queryKey: RUNS_KEY }),
  });
}

/** Compute a v2 risk score (pure calculation – no DB). */
export function useScoreV2() {
  return useMutation({
    mutationFn: (req: ScoreV2Request) => purpleApi.scoreV2(req),
  });
}

```

Module 9 — Nmap Scan Engine

Async Nmap wrapper. Runs scans as subprocesses (quick/full/stealth/udp/vuln), parses XML output, extracts open ports and OS guesses, persists scan_results rows.

Tests: 47/47 passing

Dependencies

```
nmap (system: dnf/apt install nmap) | asyncio subprocess | xml.etree.ElementTree (stdlib)
```

Bug caught by tests

SCAN_FLAGS imported from scan_result.py (model) instead of nmap_service.py — caught immediately by integration test setup errors.

Source Files

```
threatos/models/scan_result.py
```



```
# List of {"host": str, "os_guess": str, "accuracy": int}
os_guesses: Mapped[list] = mapped_column(JSONBCompat, nullable=False, default=list)

# █ Full output ████████████████████████████████████████████████████████████████████████████
scan_data: Mapped[dict] = mapped_column(JSONBCompat, nullable=False, default=dict)
error_detail: Mapped[str | None] = mapped_column(String(1000), nullable=True)

# █ Audit ████████████████████████████████████████████████████████████████████████████████
requested_by: Mapped[str | None] = mapped_column(String(100), nullable=True)

__table_args__ = (
    Index("ix_scan_results_target", "target"),
    Index("ix_scan_results_status", "status"),
    Index("ix_scan_results_started", "started_at"),
)

def __repr__(self) -> str:
    return (
        f"<ScanResult target={self.target!r} "
        f"status={self.status!r} "
        f"hosts_up={self.hosts_up}>"
    )
```

```
threatos/services/nmap_service.py
```



```

Parse Nmap XML output into a structured dict.
Returns the full parsed tree plus extracted summaries.
"""
try:
    root = ET.fromstring(xml_str)
except ET.ParseError:
    return {"error": "invalid_xml", "raw": xml_str[:500]}

hosts_up = 0
hosts_down = 0
open_ports: list[dict] = []
os_guesses: list[dict] = []
hosts_data: list[dict] = []

for host in root.findall("host"):
    status = host.find("status")
    state = status.get("state", "unknown") if status is not None else "unknown"

    if state == "up":
        hosts_up += 1
    else:
        hosts_down += 1
        continue

    # Extract address
    addr_el = host.find("address[@addrtype='ipv4']")
    if addr_el is None:
        addr_el = host.find("address")
    host_ip = addr_el.get("addr", "unknown") if addr_el is not None else "unknown"

    # Extract open ports
    ports_el = host.find("ports")
    host_ports: list[dict] = []
    if ports_el is not None:
        for port_el in ports_el.findall("port"):
            port_state = port_el.find("state")
            if port_state is None or port_state.get("state") != "open":
                continue
            svc_el = port_el.find("service")
            service = svc_el.get("name", "unknown") if svc_el is not None else "unknown"
            port_info = {
                "host": host_ip,
                "port": int(port_el.get("portid", 0)),
                "protocol": port_el.get("protocol", "tcp"),
                "service": service,
            }

```

`threatos/api/routers/scans_router.py`


```
... (showing first 100 of 257 lines)
```

[illegible]

```
threatos/tests/integration/test_scans_route.py
```



```

        new_callable=AsyncMock, return_value=NMAP_OK):
    resp = await test_client.post("/api/scans", json={
        "target": "10.0.0.1", "scan_type": "quick"
    })
    assert resp.json()["status"] == "completed"

@pytest.mark.asyncio
async def test_create_and_run_open_ports_populated(test_client: AsyncClient):
    with patch("threatos.services.nmap_service._run_nmap",
        new_callable=AsyncMock, return_value=NMAP_OK):
        resp = await test_client.post("/api/scans",
            json={"target": "10.0.0.1"})

        data = resp.json()
        assert data["hosts_up"] == 1
        assert data["hosts_down"] == 1
        assert len(data["open_ports"]) == 2

@pytest.mark.asyncio
async def test_create_and_run_response_schema(test_client: AsyncClient):
    with patch("threatos.services.nmap_service._run_nmap",
        new_callable=AsyncMock, return_value=NMAP_OK):
        resp = await test_client.post("/api/scans",
            json={"target": "10.0.0.1"})

        data = resp.json()
        required = {
            "id", "target", "scan_type", "status",
            "hosts_up", "hosts_down", "open_ports",
            "os_guesses", "duration_s",
        }
        assert required.issubset(set(data.keys()))

@pytest.mark.asyncio
async def test_create_and_run_nmap_failure_returns_failed(test_client: AsyncClient):
    with patch("threatos.services.nmap_service._run_nmap",
        new_callable=AsyncMock, return_value=NMAP_ERROR):
        resp = await test_client.post("/api/scans",
            json={"target": "10.0.0.1"})

        assert resp.status_code == 201
        data = resp.json()
        assert data["status"] == "failed"
        assert data["error_detail"] is not None

```

threatos/frontend/src/hooks/useScans.ts

```

/**
 * frontend/src/hooks/useScans.ts
 * ████████████████████████████████████████
 * React Query hooks for the Nmap scan engine API.
 * Built against proven /api/scans/* routes (435 tests passing).
 *
 * Note: POST /api/scans blocks until Nmap finishes – long-running
 * mutations. Show a loading spinner while isPending is true.
 *
 * Usage:
 *   const { data: scans } = useScans();
 *   const { mutate: scan, isPending } = useRunScan();
 */
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import {
  scansApi,
  type ScanRequest,
  type ScanStatus,
} from "@lib/api";

const SCANS_KEY = ["scans"] as const;

/** Fetch scan history. */
export function useScans(params?: { target?: string; status?: ScanStatus }) {
  return useQuery({
    queryKey: [...SCANS_KEY, params],
    queryFn: () => scansApi.list(params),
    refetchInterval: 10_000,
  });
}

/** Fetch a single scan by ID, polling until it leaves 'running' state. */
export function useScan(id: string | undefined) {
  return useQuery({
    queryKey: [...SCANS_KEY, id],
    queryFn: () => scansApi.get(id!),
    enabled: id !== undefined,
    refetchInterval: (query) => {
      const status = query.state.data?.status;
      return status === "running" || status === "pending" ? 2_000 : false;
    },
  });
}

/** Create and immediately run a scan (blocking until complete). */
export function useRunScan() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (req: ScanRequest) => scansApi.run(req),
    onSuccess: () => qc.invalidateQueries({ queryKey: SCANS_KEY }),
  });
}

/** Create a pending scan without running it. */
export function useCreateScan() {

```

```

  const qc = useQueryClient();
  return useMutation({
    mutationFn: (req: ScanRequest) => scansApi.create(req),
    onSuccess: () => qc.invalidateQueries({ queryKey: SCANS_KEY }),
  });
}

/** Execute an existing pending scan. */
export function useExecuteScan() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: ({ id, timeout }: { id: string; timeout?: number }) =>
      scansApi.execute(id, timeout),
    onSuccess: () => qc.invalidateQueries({ queryKey: SCANS_KEY }),
  });
}

```

Alembic Migrations

Run alembic upgrade head once against a live PostgreSQL on Oracle Linux 8.

```
source .venv/bin/activate
alembic upgrade head                # apply all migrations
alembic revision --autogenerate -m "add column" # after model changes
alembic downgrade -1                # roll back one step
alembic current                     # show active revision
```

threatos/migrations/versions/001_initial.py

```
"""initial schema

Revision ID: 001
Revises:
Create Date: 2025-01-01 00:00:00.000000
"""
from __future__ import annotations
from typing import Sequence, Union
from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql

revision: str = "001"
down_revision: Union[str, None] = None
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None


def upgrade() -> None:
    # raw_events - monthly range-partitioned
    op.execute("""
        CREATE TABLE IF NOT EXISTS raw_events (
            id          UUID          NOT NULL DEFAULT gen_random_uuid(),
            received_at TIMESTAMPTZ NOT NULL DEFAULT now(),
            log_source   VARCHAR(30)  NOT NULL,
            raw_payload  JSONB        NOT NULL DEFAULT '{}'::jsonb,
            normalized   JSONB,
            hash         VARCHAR(64)  UNIQUE,
            PRIMARY KEY (id, received_at)
        ) PARTITION BY RANGE (received_at)
    """)
    op.execute("CREATE INDEX IF NOT EXISTS ix_raw_events_received_at ON raw_events (received_at ...
    op.execute("CREATE INDEX IF NOT EXISTS ix_raw_events_log_source ON raw_events (log_source)" ...)

    # detection_rules
    op.execute("""
        CREATE TABLE IF NOT EXISTS detection_rules (
            id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
            name        VARCHAR(200) NOT NULL UNIQUE,
            description  VARCHAR(1000),
            author       VARCHAR(100),
            technique_id VARCHAR(20)  NOT NULL,
            tactic       VARCHAR(60)  NOT NULL,
            log_sources  JSONB        NOT NULL DEFAULT '[]'::jsonb,
            platforms   JSONB        NOT NULL DEFAULT '[]'::jsonb,
            sigma_yaml   VARCHAR(10000),
            detection_ast JSONB       NOT NULL DEFAULT '{}'::jsonb,
            severity     INTEGER      NOT NULL DEFAULT 5,
            confidence   FLOAT        NOT NULL DEFAULT 0.7,
            tags         JSONB        NOT NULL DEFAULT '[]'::jsonb,
            enabled       BOOLEAN     NOT NULL DEFAULT true,
            last_triggered TIMESTAMPTZ,
            trigger_count BIGINT      NOT NULL DEFAULT 0
        )
    """)

... (showing first 120 of 191 lines)
```

```

op.execute("CREATE INDEX IF NOT EXISTS ix_detection_rules_technique_id ON detection_rules ( ...)

# alerts
op.execute("""
    CREATE TABLE IF NOT EXISTS alerts (
        id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
        rule_id            VARCHAR(100),
        rule_name          VARCHAR(200),
        event_id           VARCHAR(100),
        technique_id       VARCHAR(20)        NOT NULL,
        tactic             VARCHAR(60)        NOT NULL,
        severity           INTEGER            NOT NULL,
        confidence         FLOAT              NOT NULL,
        asset_criticality   INTEGER            NOT NULL DEFAULT 2,
        risk_score         FLOAT              NOT NULL,
        entity_host        VARCHAR(255),
        entity_user        VARCHAR(255),
        entity_process     VARCHAR(255),
        entity_ip          VARCHAR(45),
        status             VARCHAR(20)        NOT NULL DEFAULT 'open'
                                CHECK (status IN ('open', 'investigating', 'escalated', 'closed', 'fa ...
        created_at        TIMESTAMPTZ        NOT NULL,
        updated_at         TIMESTAMPTZ        NOT NULL,
        closed_at          TIMESTAMPTZ,
        description        VARCHAR(500),
        raw_match          JSONB
    )
""")
op.execute("CREATE INDEX IF NOT EXISTS ix_alerts_technique_id ON alerts (technique_id)")
op.execute("CREATE INDEX IF NOT EXISTS ix_alerts_status ON alerts (status)")
op.execute("CREATE INDEX IF NOT EXISTS ix_alerts_risk_score ON alerts (risk_score)")

# coverage_matrix
op.execute("""
    CREATE TABLE IF NOT EXISTS coverage_matrix (
        technique_id      VARCHAR(20)        PRIMARY KEY,
        technique_name     VARCHAR(200),
        tactic            VARCHAR(60),
        rule_count         INTEGER            NOT NULL DEFAULT 0,
        confidence_avg     FLOAT              NOT NULL DEFAULT 0.0,
        covered            BOOLEAN            NOT NULL DEFAULT false,
        last_triggered     TIMESTAMPTZ,
        platforms          JSONB              NOT NULL DEFAULT '[]'::jsonb,
        priority_gap       BOOLEAN            NOT NULL DEFAULT false,
        refreshed_at       TIMESTAMPTZ
    )
""")

# assets
op.execute("""
    CREATE TABLE IF NOT EXISTS assets (
        id                VARCHAR(36)        PRIMARY KEY,
        hostname          VARCHAR(255)        NOT NULL UNIQUE,
        ip_addresses      JSONB              NOT NULL DEFAULT '[]'::jsonb,
        criticality        INTEGER            NOT NULL DEFAULT 2
                                CHECK (criticality BETWEEN 1 AND 4),
        owner_team        VARCHAR(100),
        environment       VARCHAR(30)        NOT NULL DEFAULT 'unknown',
        os_type           VARCHAR(30)        NOT NULL DEFAULT 'unknown',
        tags              JSONB              NOT NULL DEFAULT '[]'::jsonb,
        extra             JSONB              NOT NULL DEFAULT '{}'::jsonb
    )
""")
op.execute("CREATE INDEX IF NOT EXISTS ix_assets_criticality ON assets (criticality)")

```

Full Stack Deployment — Oracle Linux 8

systemd — threatos-api.service

```
[Unit]
Description=ThreatOS FastAPI
After=network.target docker.service

[Service]
Type=exec
User=threatos
WorkingDirectory=/opt/threatos
Environment="PATH=/opt/threatos/.venv/bin:/usr/local/bin:/usr/bin"
ExecStart=/opt/threatos/.venv/bin/uvicorn threatos.main:app \
    --host 0.0.0.0 --port 8000 --workers 4
Restart=always

[Install]
WantedBy=multi-user.target
```

systemd — threatos-worker.service

```
[Unit]
Description=ThreatOS ingest worker
After=threatos-api.service

[Service]
Type=exec
User=threatos
WorkingDirectory=/opt/threatos
ExecStart=/opt/threatos/.venv/bin/python -m threatos.workers.ingest_worker
Restart=always

[Install]
WantedBy=multi-user.target

# Enable:
# sudo systemctl daemon-reload
# sudo systemctl enable --now threatos-api threatos-worker
```

.env (production)

```
APP_ENV=production
DATABASE_URL=postgresql+asyncpg://threatos:secret@localhost:5432/threatos
REDIS_URL=redis://localhost:6379/0
REDIS_CONSUMER_GROUP=detection-workers
WORKER_BATCH_SIZE=500
ASSET_CRITICALITY_TTL=300
COVERAGE_REFRESH_INTERVAL_SECONDS=3600
NMAP_BINARY_PATH=/usr/bin/nmap
```

nginx WebSocket proxy

```
location /api/ { proxy_pass http://127.0.0.1:8000; }
location /ws/ {
    proxy_pass http://127.0.0.1:8000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_read_timeout 86400;
}
```

ThreatOS v2

Modules	9 (Ingest · Rules · Alerts · ATT&CK · Assets · Worker · Chains · Purple · Nmap)
Tests	435 / 435 passing — zero failures across all modules
DB tables	8 (raw_events partitioned · detection_rules · alerts · coverage_matrix · assets · attack_chains · purple_team_runs · s
Workers	2 (ingest_worker: Redis XREADGROUP consumer / coverage_worker: APScheduler)
WebSocket	/ws/alerts → Redis pubsub → browser via useAlertStream.ts
Frontend	9 React Query hooks + 1 WebSocket hook + typed api.ts
Target OS	Oracle Linux 8 / Python 3.11 / docker compose
